

Utilização de múltiplos componentes VHDL – Tutorial

1. Objetivo:

Este tutorial irá guiá-lo pelos procedimentos básicos para utilização de componentes na linguagem VHDL, mostrando como fazer simulações nesses casos, por meio tanto do Quartus, como do LogiSim-evolution.

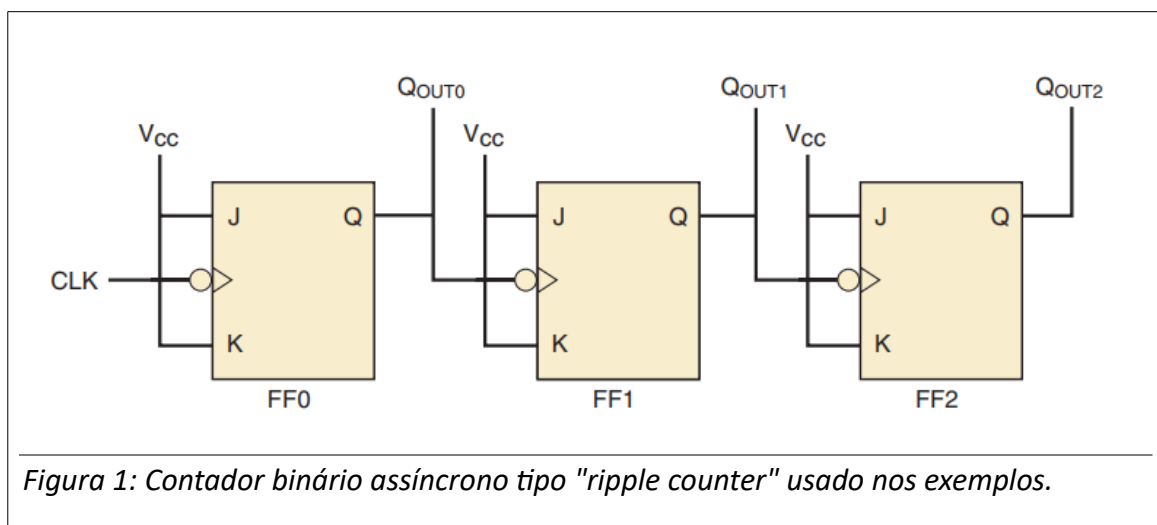
O projeto utilizado como referência foi obtido do Capítulo 5, seção 5.29 – “HDL Digital Circuits with Multiple Components” do livro “Digital Systems – Principles and Applications – 12th ed.” de Ronal J. Tocci, Neal S. Widmer e Gregory L. Moss.

Diversas alternativas serão trabalhadas com o mesmo projeto a saber:

- No Quartus:
 - Um único arquivo VHDL contendo o Nível Superior e o Componente.
 - Arquivos VHDL separados para Nível Superior e Componente.
 - Declaração do componente em um *package* VHDL, referenciado pelo Nível Superior.
- No LogiSim-evolution, em um único arquivo de projeto *.circ* (versão 3.6.1):
 - Um único circuito, contendo uma única entidade VHDL com o Nível Superior e o Componente.
 - Circuitos separados e entidades VHDL separadas para Nível Superior e Componente.
 - Circuitos separados e entidades VHDL separadas para *package* VHDL com a declaração do componente, Nível Superior e Componente.

2. O Projeto

A Figura 1 abaixo ilustra um contador binário assíncrono do tipo “*ripple counter*” com uma entrada (clk) e três bits de saída (qout2..0).



Nesse exemplo, o nível superior será o próprio contador, o qual fará uso 3 vezes do componente flip-flop tipo JK com resposta a flanco negativo do sinal de relógio.

O Componente consiste em um flip-flop tipo JK com três entradas (clk, j e k) e uma única saída (q). A Figura 2 a seguir mostra o código fonte em VHDL, inserido em um único arquivo, como sugerido no exemplo do livro do Tocci. Na figura foram omitidas as declarações das bibliotecas utilizadas, que precisarão ser inseridas ou o arquivo não compilará.

```
1  ENTITY fig5_86 IS
2  PORT (   clock           :IN BIT;
3          qout             :BUFFER BIT_VECTOR (2 DOWNTO 0));
4  END fig5_86;
5  ARCHITECTURE counter OF fig5_86 IS
6      SIGNAL high          :BIT;
7      COMPONENT neg_jk
8      PORT (   clk, j, k    :IN BIT;
9              q             :OUT BIT);
10     END COMPONENT;
11 BEGIN
12     high <= '1';          -- connect to Vcc
13     ff0: neg_jk PORT MAP (j => high, k => high, clk => clock, q => qout(0));
14     ff1: neg_jk PORT MAP (j => high, k => high, clk => qout(0), q => qout(1));
15     ff2: neg_jk PORT MAP (j => high, k => high, clk => qout(1), q => qout(2));
16 END counter;
17
18 ENTITY neg_jk IS
19 PORT (   clk, j, k        :IN BIT;
20         q                 :OUT BIT);
21 END neg_jk;
22 ARCHITECTURE simple OF neg_jk IS
23     SIGNAL qstate         :BIT;
24 BEGIN
25     PROCESS (clk)
26     BEGIN
27         IF (clk'EVENT AND clk = '0') THEN
28             IF j = '1' AND k = '1' THEN qstate <= NOT qstate; -- toggle
29             ELSIF j = '1' AND k = '0' THEN qstate <= '1';      -- set
30             ELSIF j = '0' AND k = '1' THEN qstate <= '0';      -- reset
31             END IF;
32         END IF;
33     END PROCESS;
34     q <= qstate           -- connect flip-flop state to output
35 END simple;
```

Nível Superior ("Top Level")

Componente

Figura 2: Código fonte VHDL para Nível Superior e Componente – simplificado.

No bloco do Nível Superior, na região declaratória de sua arquitetura (essa é a região após a palavra "Architecture" e antes da palavra "Begin" – linhas 6 a 10), vemos duas declarações: um sinal denominado "high" e um componente, denominado "neg_jk" com suas definições de entradas (clk, j e k) e saídas (q).

Entre as diretivas "Begin" e "End" da arquitetura do nível superior são utilizados 3 flip-flops do tipo "neg_jk", que recebem os rótulos de FF0, FF1 e FF2, e cujas interconexões estão definidas com as diretivas "Port Map" (linhas 13 a 15). Em linguagem VHDL dizemos que os flip-flops foram **instanciados** no nível superior com as diretivas "Port Map".

A Figura 3 mostra o mesmo arquivo, agora com as declarações das bibliotecas (uma para cada entidade), o qual pode ser compilado e simulado sem problemas no Quartus e também no Logisim-evolution. Os FF's FF0 a FF2 foram instanciados com "Port Map" nas linhas 24 a 26. Na diretiva "Port Map" o símbolo ">=" deve ser lido como "vai ligado em". Por exemplo: "clk => clock" deve ser interpretado como "a porta clk do flip-flop vai ligada no sinal clock".

Embora o Quartus opere sem problemas com o tipo "bit", os tipos foram alterados para

“std_logic”, que é o único tipo aceito nas *entidades* do Logisim-evolution.

A Figura 4 mostra o resultado da simulação, em que o contador contou de 0 a 7. Este projeto pode ser encontrado na pasta denominada “arq_unico” no conjunto multiplos_vhdl.zip de projetos que acompanha este tutorial.

```
1  ----- Nivel Superior -----
2  library ieee;
3  use ieee.std_logic_1164.all;
4  -----
5  entity ripple_cnt_jk is
6  port(
7      clock : in std_logic;           -- entrada de relógio
8      qout  : out std_logic_vector(2 downto 0) -- output vector example
9  );
10 end ripple_cnt_jk;
11 -----
12 architecture contador of ripple_cnt_jk is
13     signal high: std_logic := '1';
14     signal qstate: std_logic_vector(2 downto 0);
15     component neg_jk is
16     port(
17         clk, j, k : in std_logic;
18         q         : out std_logic
19     );
20     end component;
21
22 begin
23
24     ff0: neg_jk PORT MAP (j => high, k => high, clk => clock, q => qstate(0));
25     ff1: neg_jk PORT MAP (j => high, k => high, clk => qstate(0), q => qstate(1));
26     ff2: neg_jk PORT MAP (j => high, k => high, clk => qstate(1), q => qstate(2));
27
28     qout <= qstate;
29 end contador;
30 ----- Componente -----
31 library ieee;
32 use ieee.std_logic_1164.all;
33 -----
34 entity neg_jk is
35     port(
36         clk, j, k : in std_logic;
37         q         : out std_logic
38     );
39 end neg_jk;
40 -----
41 architecture cnt of neg_jk is
42
43     signal qstate : std_logic := '0';
44     begin
45         process(clk)
46         begin
47             if (clk'event and clk = '0') then
48                 if j = '1' and k = '1' then qstate <= not qstate;
49                 elsif j = '1' and k = '0' then qstate <= '1';
50                 elsif j = '0' and k = '1' then qstate <= '0';
51                 end if;
52             end if;
53         end process;
54         q <= qstate;
55     end cnt;
56 -----
57
```

Figura 3: Arquivo modificado para compilação e simulação no Quartus

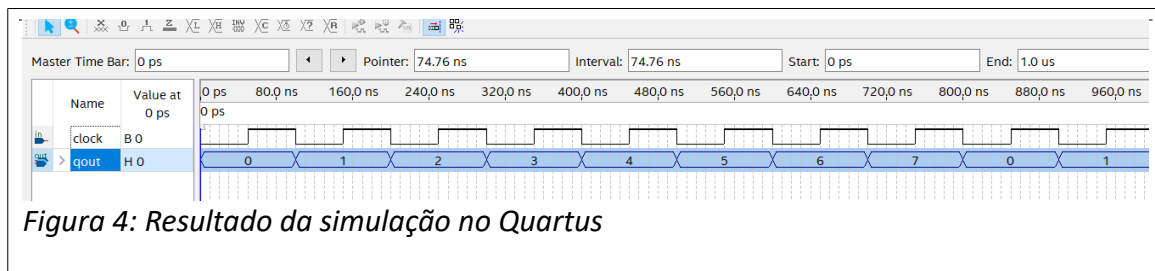


Figura 4: Resultado da simulação no Quartus

Exatamente o mesmo código pode ser utilizado no Logisim-evolution (testado na versão 3.6.1) como exemplificado no projeto `arq_unico_circ.circ`, contido na pasta `arq_unico_circ` no conjunto `multiplos_vhdl.zip` de projetos que acompanha este tutorial. Basta criar uma entidade VHDL no Logisim-evolution e utilizá-la como um bloco no circuito.

3. Arquivos separados para Nível Superior e Componente(s)

Nesta versão as definições do contador no Nível Superior e do Componente flip-flop tipo JK serão acomodadas em arquivos separados.

No exemplo utilizado temos um único componente, mas nada impede que tenhamos mais do que um, todos declarados como componente no nível superior e cada um com a definição de sua arquitetura em um arquivo em separado.

A vantagem desta versão é que os arquivos de componentes, estando cada qual em um arquivo isolado dos demais, podem ser reutilizados em outros projetos simplesmente copiando-os para as áreas dos outros projetos. Essa configuração pode ser também implementada no LogiSim-evolution, como veremos adiante.

A Figura 5 mostra o arquivo do componente flip-flop:

```

1  -----
2  library ieee;
3      use ieee.std_logic_1164.all;
4  -----
5  entity neg_jk is
6      port(
7          clk, j, k : in std_logic; -- entrada do clock
8          q         : out std_logic -- saída do flip-flop
9      );
10 end neg_jk;
11 -----
12 architecture cnt of neg_jk is
13     signal qstate : std_logic := '0';
14
15 begin
16     process(clk)
17     begin
18         if (clk'event and clk = '0') then
19             if j = '1' and k = '1' then qstate <= not qstate;
20             elsif j = '1' and k = '0' then qstate <= '1';
21             elsif j = '0' and k = '1' then qstate <= '0';
22             end if;
23         end if;
24     end process;
25     q <= qstate;
26 end cnt;
27
28 -----

```

Figura 5: Arquivo VHDL do componente flip-flop JK

Nesse arquivo, o tipo das variáveis na entidade foi novamente alterado para “std_logic”, para ficar compatível com as restrições do LogiSim-evolution.

Alterações semelhantes foram necessárias no caso do arquivo contendo a descrição VHDL do nível superior, como pode ser visto na Figura 6. Como o LogiSim-evolution não suporta o tipo “buffer”, que é um tipo de saída que pode ser lida internamente, foi preciso fazer uso da variável “temp” (definida na linha 14) para esse fim, a qual depois é atribuída à saída (na linha 27).

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  -----
4  entity ripple_cnt_jk is
5  port(
6      clock : in  std_logic;           -- entrada de relógio
7      gout  : out std_logic_vector(2 downto 0) -- saída do contador
8  );
9  end ripple_cnt_jk;
10 -----
11 architecture contador of ripple_cnt_jk is
12     constant high: std_logic := '1';
13     signal temp: std_logic_vector(2 downto 0);
14     component neg_jk is               -- declaração de componente
15     port(                             -- definido em outro arquivo
16         clk, j, k : in  std_logic;
17         q         : out std_logic
18     );
19     end component;
20
21 begin
22
23     ff0: neg_jk PORT MAP (j => high, k => high, clk => clock, q => temp(0));
24     ff1: neg_jk PORT MAP (j => high, k => high, clk => temp(0), q => temp(1));
25     ff2: neg_jk PORT MAP (j => high, k => high, clk => temp(1), q => temp(2));
26
27     gout <= temp;
28
29 end contador;
30 -----
31
```

Figura 6: Código VHDL para o Nível Superior do projeto dois_arq.

Este projeto para o Quartus pode ser encontrado na pasta denominada “dois_arq” no conjunto de projetos multiplos_vhdl.zip que acompanha este tutorial. Os resultados de sua simulação no Quartus são os mesmos que os já apresentados na Figura 4 anterior.

Exatamente o mesmo projeto, com os mesmos códigos da Figura 6 para o nível superior (com uma pequena adaptação) e da Figura 5 para o componente, pode ser simulado no LogiSim-evolution.

O circuito exemplo para o LogiSim-evolution está no arquivo “dois_arq_circ” na pasta de mesmo nome no conjunto de projetos multiplos_vhdl.zip que acompanha este tutorial.

A pequena adaptação a ser realizada no circuito de nível superior no caso do LogiSim-evolution consiste em alterar o nome do componente de “neg_jk” para “neg_jk” seguido de *underline*, seguido do nome (“label”) atribuído ao componente no LogiSim-evolution. Por exemplo, se atribuirmos o nome “ffjk” ao bloco contendo o componente, no nível superior teremos que utilizar “neg_jk_ffjk” em lugar de simplesmente “neg_jk” como na Figura 6.

O único cuidado a ser tomado portanto, no caso do LogiSim-evolution, é com o nome do componente no bloco de nível superior. No exemplo anterior, foi utilizado o nome “neg_jk_ffjk”, que é o nome da “entity” do componente concatenado com seu “label”. Para que isso funcione da forma como está, o bloco HDL que contém o componente no LogiSim-evolution terá de receber um rótulo-

lo (“*label*”). Caso o componente seja deixado sem rótulo, o Logisim-evolution criará um nome internamente, não acessível ao usuário. Essa versão pode ser encontrada no arquivo “dois_arq_circ_v1.circ” disponível na pasta dois_arq_circ no conjunto multiplos_vhdl.zip de projetos que acompanha este tutorial, onde foram criados dois circuitos no Logisim-evolution, um para acomodar o componente e outro para o contador que faz uso desse componente.

Essa versão tem o inconveniente de exigir que a entidade vhdl que utiliza o componente neg_jk renomeie o componente para neg_jk_ffjk. Caso haja várias outras entidades vhdl fazendo uso do mesmo componente, todas precisarão sempre usar o componente renomeado, ficando diferentes dos arquivos utilizados no Quartus. Existe, porém, uma alternativa que permite fazer uso do componente sem necessidade de renomeá-lo. Se dentro de uma entidade vhdl do Logisim-evolution houver mais de uma entidade (semelhante ao exemplo anterior, de um único arquivo), o Logisim-evolution renomeará apenas a primeira entidade, preservando as demais (os componentes) com o nome inalterado.

Essa alternativa está exemplificada no arquivo “dois_arq_circ_v2.circ”, onde foi criada uma entidade de nome “comps”, que não faz nada (e que será renomeada pelo Logisim-evolution) e, junto com essa entidade foi inserido o componente “neg_jk”, cujo nome agora será preservado. Fazendo dessa maneira, todas as demais entidades – caso existam – que venham a fazer uso do componente “neg_jk”, poderão utilizá-lo com o nome original, sem necessitar renomeá-lo, mantendo os arquivos idênticos aos utilizados depois no Quartus.

Atenção: o circuito “comps” deve ser simulado antes que o(s) seu(s) componente(s) possam ser utilizados em outros circuitos do mesmo projeto, para que o Logisim-evolution crie uma biblioteca com esse(s) componente(s).

O arquivo “dois_arq_circ_v2.circ” está disponível na pasta dois_arq_circ no conjunto multiplos_vhdl.zip de projetos que acompanha este tutorial.

4. Declaração de componente em um “*package*” VHDL

Até aqui consideramos duas alternativas de utilização de componentes vhdl:

- Definição (com *library*, *entity* e *architecture*), declaração (com *component*) e instanciamento (com *port map*) em um único arquivo (ou uma única entidade vhdl do Logisim-evolution).
- Definição (com *library*, *entity* e *architecture*) em um arquivo (ou uma entidade vhdl do Logisim-evolution) e declaração (com *component*) e instanciamento (com *port map*) em outro arquivo (ou outra entidade vhdl do Logisim-evolution).

Uma terceira alternativa permite isolar também a declaração do componente em um arquivo vhdl separado, o qual agrega uma ou mais declarações de componentes usando a palavra-chave “*package*”. Esse arquivo, e mais os vários arquivos com as definições dos componentes declarados no “*package*”, são então colocados, para utilização pelo Quartus, em uma “*library*”, que nada mais é do que um diretório contendo todos os arquivos de componentes e o arquivo “*package*”.

No caso do Logisim-evolution, o código do “*package*” é inserido como uma entidade vhdl e então utilizado em um circuito separado dentro do mesmo arquivo .circ. Também neste caso a entidade vhdl deverá conter uma primeira entidade com um nome qualquer, como na versão v2 do exemplo de dois arquivos anterior, para evitar que o “*package*” seja renomeado pelo Logisim-evolution. Um exemplo de como isso pode ser feito está contido no arquivo arq_e_package.circ da pasta arq_e_package contida no conjunto multiplos_vhdl.zip de projetos que acompanha este tutorial.

```

library ieee;
use ieee.std_logic_1164.all;
package meus_comps is

    constant high: std_logic := '1';

    component neg_jk is
        port(
            clk, j, k : in std_logic; -- entradas j,k e clock
            q          : out std_logic -- saída do flip-flop
        );
    end component;
end meus_comps;

```

Figura 7: Conteúdo do arquivo *meu_package* contendo o *package meus_comps*.

A Figura 7 acima mostra o conteúdo do arquivo chamado ***meu_package.vhd***, o qual contém o *package meus_comps*, que por sua vez contém uma única declaração de componente (embora pudesse conter diversas outras). A Figura 8 abaixo mostra o nível superior fazendo uso do *package*.

```

library ieee;
use ieee.std_logic_1164.all;
use work.meus_comps.all; -- usa todos os components de meu_package
-----
entity ripple_cnt_jk is
    port(
        clock : in std_logic; -- entrada de relógio
        qout  : out std_logic_vector(2 downto 0) -- saída do contador
    );
end ripple_cnt_jk;
-----
architecture contador of ripple_cnt_jk is
    signal temp: std_logic_vector(2 downto 0);
begin

```

Figura 8: Arquivo do nível superior fazendo uso do *package meus_comps*.

Note-se, no arquivo da Figura 8, que ele não contém mais as declarações para o componente ***neg_jk*** e para a constante ***high***. Isso porque essas declarações agora serão incluídas via o “*package*” ***meus_comps*** declarado com a diretiva *use*.

Para que o Quartus faça uso do *package meus_comps* é preciso:

1. Incluir os arquivos ***meu_package.vhd*** e ***neg_jk.vhd*** no projeto do Quartus (além é claro do arquivo com o nível superior ***ripple_cnt_jk.vhd***).
2. Informar para o Quartus o caminho onde os arquivos ***meu_package.vhd*** e ***neg_jk.vhd*** se encontram, visto que foram acomodados em um diretório separado. Ver Figura 9 abaixo.

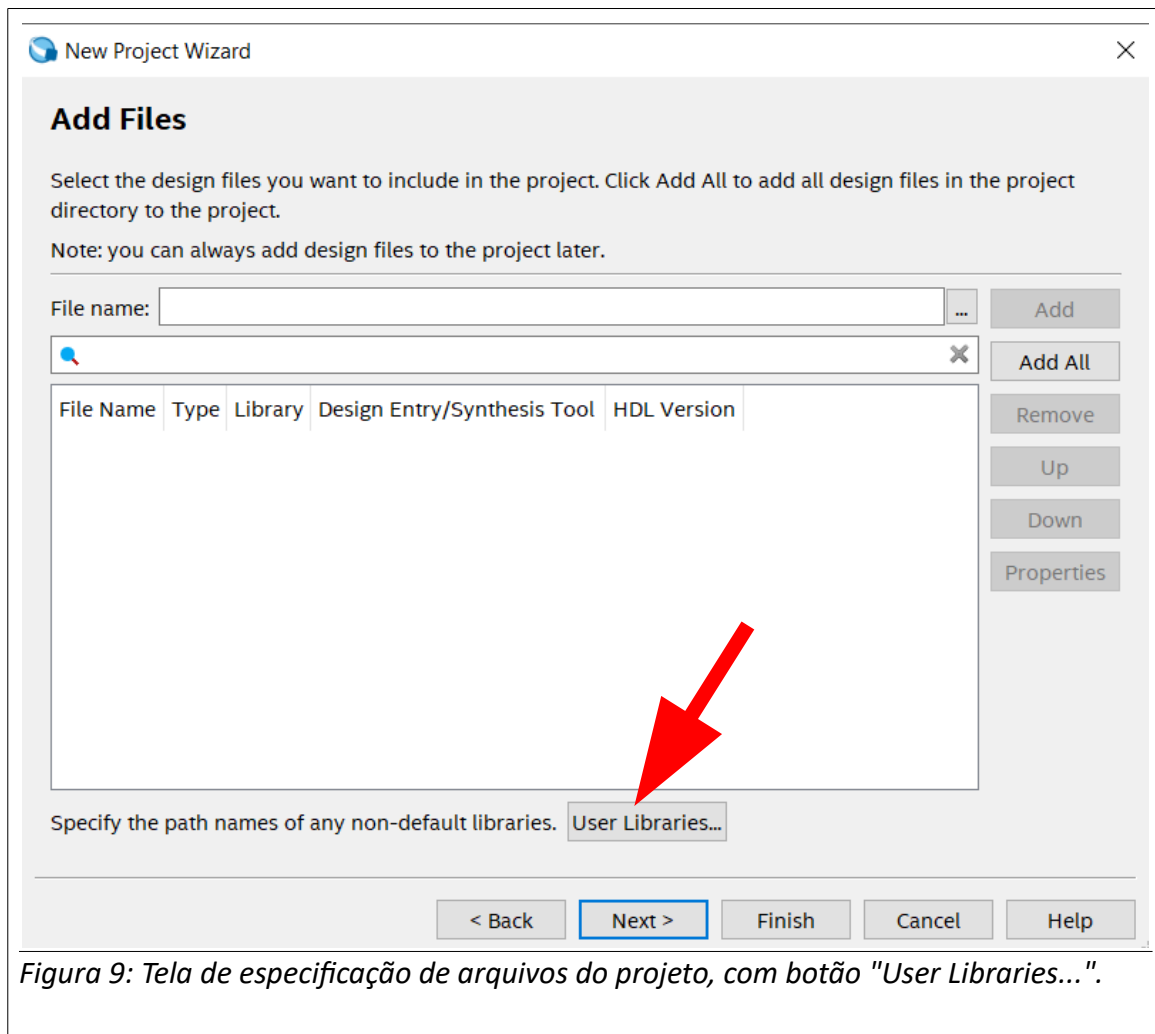
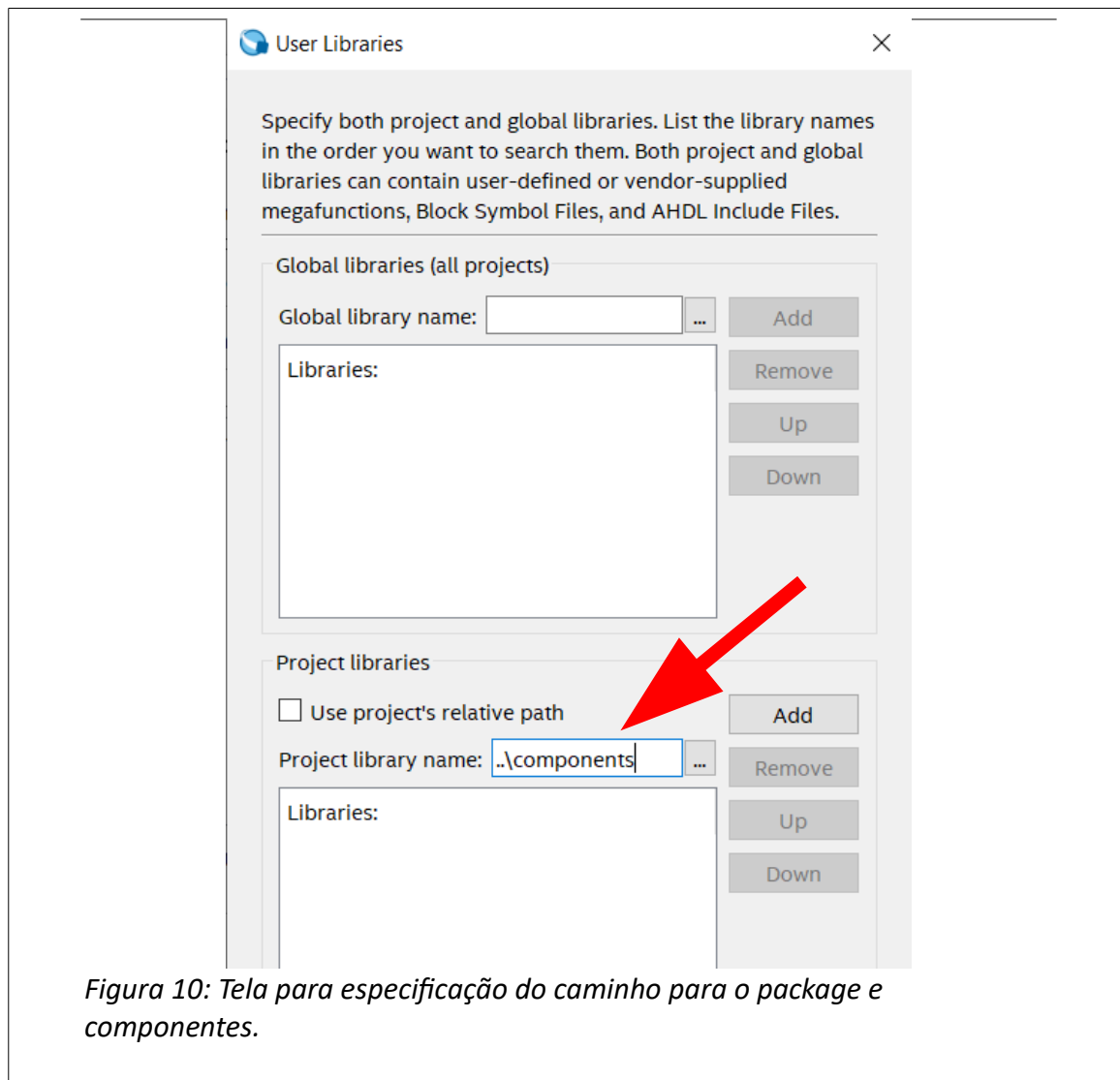


Figura 9: Tela de especificação de arquivos do projeto, com botão "User Libraries...".

Na Figura 9 acima, a tela utilizada para adicionar arquivos ao projeto é mostrada. Nessa tela, deve-se usar o botão "User Libraries..." para entrar na tela onde o caminho do diretório contendo o *package* e seus componentes poderá ser especificado. Ver Figura 10 a seguir.



Este projeto pode ser encontrado na pasta denominada “**arq_e_package**” no conjunto multiplos_vhdl.zip de projetos que acompanha este tutorial. O *package* e seu componente foram colocados na pasta **componentes**.

Uma última observação: no Logisim-evolution também é possível operar com bibliotecas e *packages*. Porém, como o Logisim-evolution faz uso do ModelSim, é preciso saber criar as correspondentes bibliotecas (*libraries*) no Logisim-evolution, o que não será abordado neste tutorial.

5. Packages com Functions

Uma *Function* em VHDL é semelhante a uma função “*inline*” em linguagem C: sua declaração tem um nome, uma lista de parâmetros formais com a especificação de tipos e um valor de retorno, também com a especificação de tipo. Sua definição congrega uma série de instruções a serem realizadas com os parâmetros e o resultado é então retornado. A *function* pode então ser chamada a partir de algum código VHDL. *Functions* são bastante usadas para conter ações repetitivas ou conversões de tipos.

A Figura 11 a seguir contém um exemplo de declaração de uma *function* em VHDL, que recebe um *signal* do tipo *std_logic_vector* e retorna outro valor, também do tipo *std_logic_vector*. Como o nome da *function* sugere, ela faz a conversão de um vetor binário para display de sete segmentos.

```

=====
-- Função para decodificação de display 7 segmentos
function decode_7s(signal count: std_logic_vector) return std_logic_vector;
=====

```

Figura 11: Declaração da *function decode_7s* em VHDL

Um *package* VHDL que contenha a declaração de uma *function*, precisa também conter uma segunda parte, chamada *package body*, na qual a *function* em questão é definida. A Figura 12 a seguir mostra o esquema geral de um *package* contendo uma função e seu *package body* com a definição dessa função.

```

library ieee;
use ieee.std_logic_1164.all;
package components is

    component dff_vhdl is
        -- .....
    end component;
    -- outras declarações de components
=====
    function decode_7s ....
    -- outras declarações de functions
=====
end components;

package body components is

    function decode_7s..... is
        -- declarações de variables aqui
    begin
        -- ... comandos executados pela function aqui
    end decode_7s;

end components;

```

Figura 12: Esquema geral de *package* com *package body*.

A Figura 13 a seguir mostra um exemplo concreto de implementação de *package body* com a definição da *function decode_7s*.

```

package body crono_comps is
  function decode_7s(signal count: std_logic_vector) return std_logic_vector is
    variable v_in: std_logic_vector(3 downto 0) := count;
    variable result: std_logic_vector(6 downto 0) := "1111111";
  begin
    case v_in is
      when "0000" => result := "1111110";      -- 0 - 7Eh
      when "0001" => result := "0110000";      -- 1 - 30h
      when "0010" => result := "1101101";      -- 2 - 6Dh
      when "0011" => result := "1111001";      -- 3 - 79h
      when "0100" => result := "0110011";      -- 4 - 33h
      when "0101" => result := "1011011";      -- 5 - 5Bh
      when "0110" => result := "1011111";      -- 6 - 5Fh
      when "0111" => result := "1110000";      -- 7 - 70h
      when "1000" => result := "1111111";      -- 8 - 7fh
      when "1001" => result := "1111011";      -- 9 - 7Bh
      when "1010" => result := "1110111";      -- A - 77h
      when "1011" => result := "0011111";      -- b - 1fh
      when "1100" => result := "1001110";      -- C - 4eh
      when "1101" => result := "0111101";      -- d - 3dh
      when "1110" => result := "1001111";      -- E - 4fh
      when "1111" => result := "1000111";      -- F - 47h
      when others => null;
    end case;
    return result;
  end decode_7s;
end crono_comps;

```

Figura 13: Implementação da definição de `decode_7s` no `package body`.

A vantagem da utilização de *functions* é que permite tornar o código VHDL principal mais claro e menos repetitivo, além de minimizar erros de digitação. Por exemplo, se um determinado código VHDL precisar acionar 4 conjuntos de saídas para acionar 4 *displays* de 7 segmentos a partir de 4 vetores binários, esse código simplesmente faria 4 chamadas à *function decode_7s* como abaixo:

```

vet_7s_01 <= decode_7s(vet_bin_01);
vet_7s_02 <= decode_7s(vet_bin_02);
vet_7s_03 <= decode_7s(vet_bin_03);
vet_7s_04 <= decode_7s(vet_bin_04);

```

No exemplo acima, embora a *function decode_7s* tenha sido definida em um único local, não há economia de recursos de hardware, cada uma das chamadas acima implementa sua versão do decodificador de forma semelhante ao uso de funções “*inline*” na linguagem C. No entanto, a intenção do código fica muito mais clara do que se os comandos do decodificador fossem listados 4 vezes. Além disso, quaisquer alterações necessárias podem ser efetuadas de maneira centralizada, alterando-se apenas a implementação do *package*.

6. Conclusão

Este tutorial ilustrou as diversas possibilidades de se reutilizar componentes VHDL, realizando simulações tanto no Logisim-evolution como no Quartus.

Para pequenos projetos, com poucos componentes, qualquer das alternativas abordadas satisfaz as necessidades. Entretanto, à medida que os projetos aumentam em complexidade e fazem uso de uma quantidade cada vez maior e mais variada de componentes, a alternativa com a separação da definição e declaração de componentes (por meio de “*packages*”) em um diretório separado é a mais indicada, pois permite a reutilização de componentes em diversos projetos diferentes.